Warmup Question

① Given an integer array A and an integer k.

return the kth largest element in A.

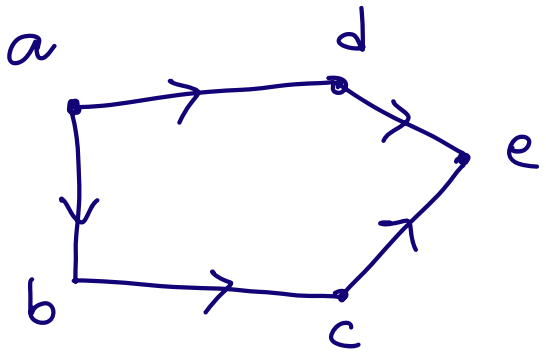
Shortest Path Problem

- Single Source Shortest Paths
  - Dijkstra's algorithm [greedy]
  - Bellman-Ford algorithm [DP]
- All-Pairs Shortest Paths
  - Floyd-Warshall algorithm [DP]

Unweighted directed/undirected graphs

let $G = (V, E)$ be a graph.

The **Shortest Path** blw two vertices is a Path with the shortest length (least # of edges)



The shortest path blw a & e is

$$P = a, d, e$$

$$\text{length}(P) = 2$$

Weighted Graphs

Let $G = (V, E)$ be a weighted digraph, with weight function $\omega : E \longrightarrow \mathbb{R}$.

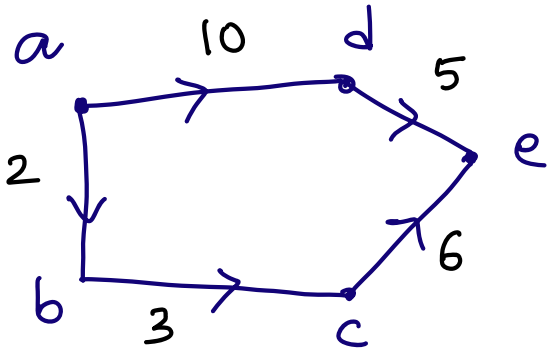
The length of the a path $P = v_0, v_1, \ldots v_k$ is the sum of the weights of its constituent edges.

$$\text{length}(P) = \sum_{i=1}^{k} \omega_{v_{i-1} v_i}$$

or

$$\omega(P) = \sum_{i=1}^{k} \omega_{v_{i-1} v_i}$$

Example



The Shortest Path b|w a and e is P: a, b, c, e

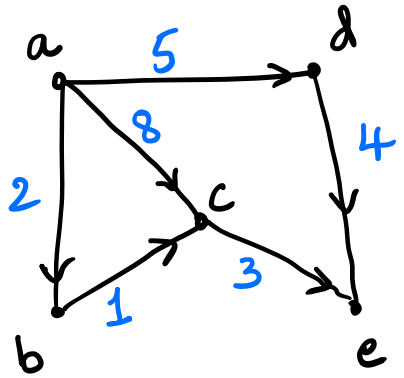
$$\text{length}(P) = \omega_{ab} + \omega_{bc} + \omega_{ce}$$

$$= 2 + 3 + 6$$

$$= 11$$

The distance from u to v, denoted $\delta(u,v)$ is the length of the minimum length Path, if there is a Path from u to v; and is ∞ otherwise.

Example:



length of the path $a, c, e = 11$

distance from a to $e = \delta(a,e) = 6$

↓

length of the Shortest a to e Path.

Formally

The **Shortest-Path weight** $\delta(u,v)$ from u to v defined by

$$\delta(u,v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{if there is a Path from } u \text{ to } v \\ \infty & \text{Otherwise} \end{cases}$$

A Shortest Path from Vertex u to Vertex v is defined as any path p with weight $w(p) = \delta(u,v)$

Problem

Single-Source Shortest Paths

I/p: Directed edge weighted graph G and a source vertex s

O/p: Find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$.

ie, For each $v \in V$, find $\delta(s, v)$ (or $dist(s, v)$)

Applications

- Social Network : Finding the shortest path between two persons (degree of separation)

  Eg: In Linkedin, they Provide some person is $2^{nd}/3^{rd}$ connection, here 2/3 is the shortest path from you to that person in the linkedin network (unweighted)

- Word ladder Puzzles:

  YES – YET – GET – GOT – GO – NO

  Given a starting word and an ending word, design an algorithm to transform one word into the other by changing, adding or deleting exactly one letter at a time, with the result being a valid English word at each step.

- Transport finished Product from factory (Single source) to all retail outlets.
- Courier company delivers items from distribution center to addressees.
- Google maps uses a version of Dijkstra's algorithm.

Back to the Problem

Initial Ideas are welcome.

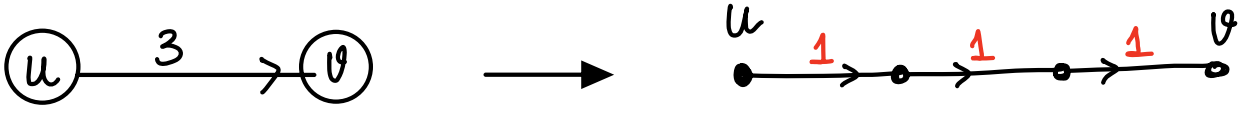
Why Not Use Breadth-First Search?

Recap: BFS computes shortest paths from a source vertex.

Works on unweighted graphs (i.e., weight of an edge is 1)

Idea:

Replace each edge e by a directed path of length w_e.



- The major problem with the above idea is that it blowsup the size of the graph.

- The length of an edge could be much bigger than the number of vertices and edges in the original graph.

- Breadth-first search would run in time linear in the size of the expanded graph, but this is not necessarily close to linear time in the size of the original graph.

Assumptions

① We assume that $\forall v \in V$, there is a s to v Path.

② Edge weights are non-negative.

ie, $w_e \geq 0 \quad \forall e \in E(G)$.

IMPORTANT

(In some applications negative edge lengths are relevant)

More on this in the next lecture.

Problem

Single-Source Shortest Paths

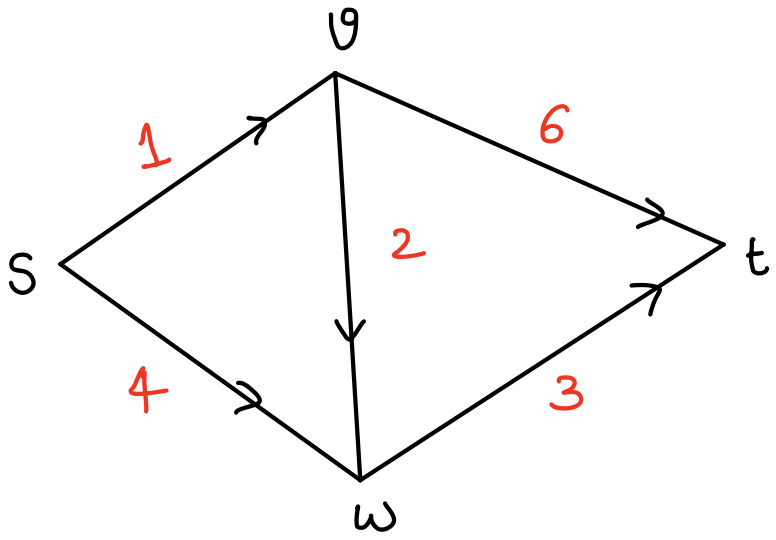
[this lecture]

I/p: Directed edge weighted graph G and a source vertex s, where edge weights are non-negative.

O/p: Find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$.

ie, For each $v \in V$, find $\delta(s,v)$ (or $dist(s,v)$)

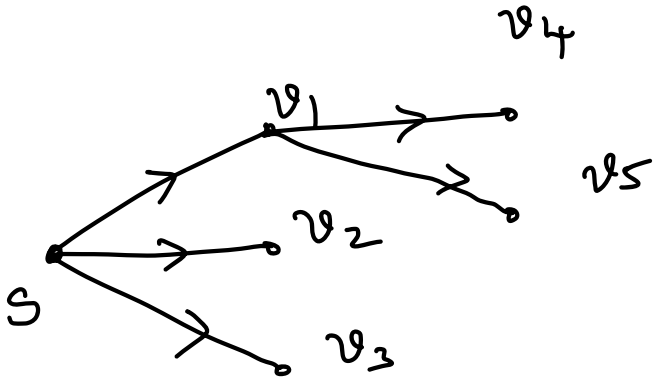
Example



For the above graph, with starting vertex s,

The shortest-path distances to s, v, w and t are 0, 1, 3, 6 .

Warmup - Ideas



Overview of the Algorithm

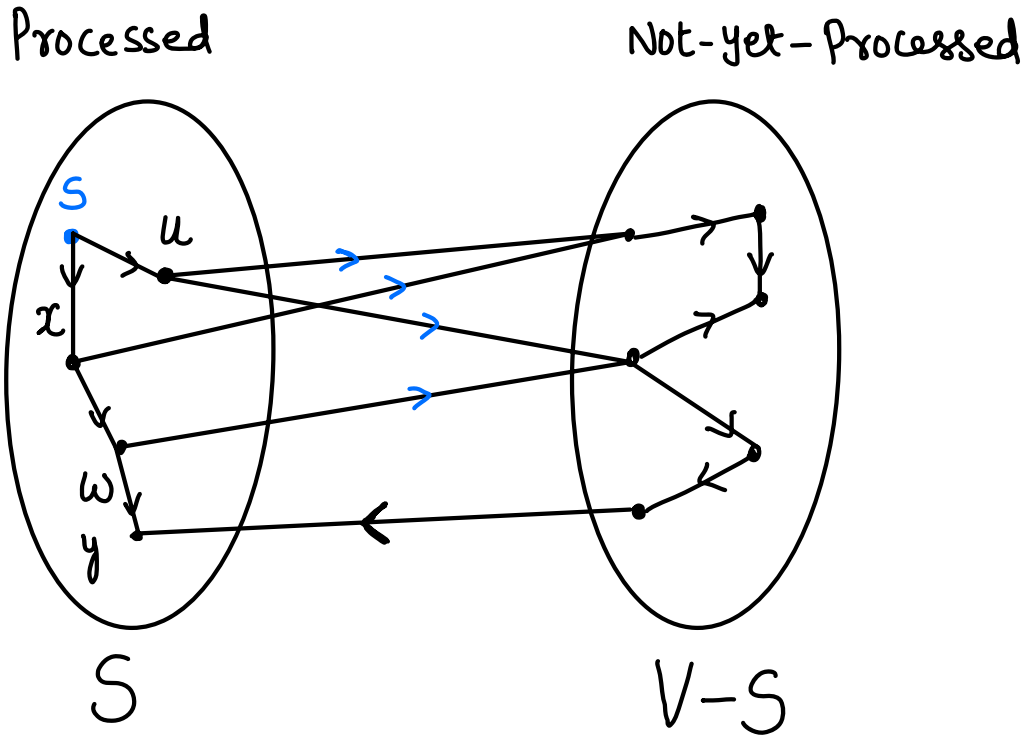
S : vertices processed so far and $d(u)$ denote $\delta(s,u)$

Initialization:

$S = \{s\}$ $\quad d(s) = 0$

$d(v) = \infty$ for every $v \neq s$

Main Step:



Greedy Choice

- Among all edges $e=uv$ with $u \in S$ and $v \in V-S$

Pick the one that minimizes

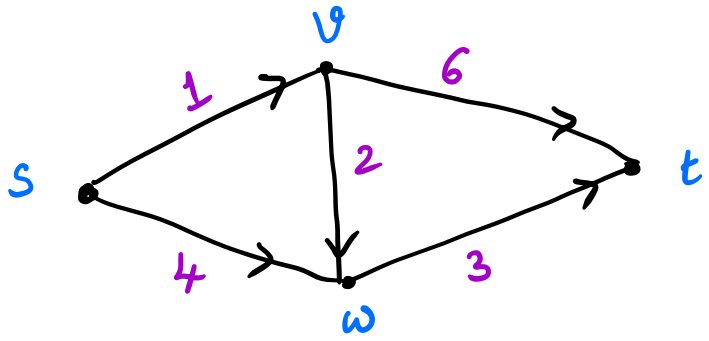
$$d(u) + w_e$$

[Say $e^* = u^*v^*$ is the edge which minimizes]

- Add v^* to S

$$d(v^*) = d(u^*) + w_{e^*}$$

Example:



Dijkstra (G, w, s)

1 Let S be the set of explored vertices

2 for each $u \in S$, we store a distance $d(u)$

3 Initially $S = \{s\}$ and $d(s) = 0$

4 While $S \neq V$

5 Select a node $v \notin S$ with at least one edge from S for which

6 $$d'(v) = \min_{\substack{e = uv \\ u \in S}} d(u) + w_{uv}$$ is as small as possible

7 Add v to S, define $d(v) = d'(v)$

Running time:

While loop runs for $n-1$ iterations as each iteration adds a new node v to S.

Selecting a correct node v:

For each node $v \notin S$, we have to go through all the edges b/w S and v to determine the minimum $\min_{\substack{e=uv \\ u \in S}} d(u) + w_{uv}$.

This takes $O(m)$ time.

So, the algorithm runs in $O(mn)$ time.

Can we do better than Naive implementation?

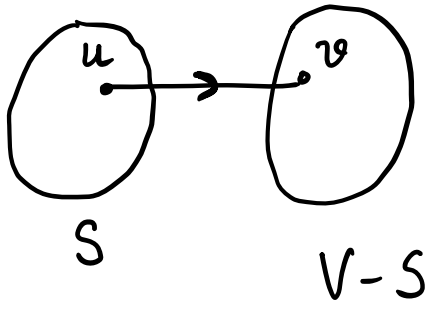
YES, Using Priority Queue

HEAP DATA STRUCTURE

We Put the vertices of V in a $\overset{\min}{}$ Priority queue with $d'(v)$ as the key for $v \in V$.

To select the node v that should be added to the set S, we need the EXTRACTMIN Operation.

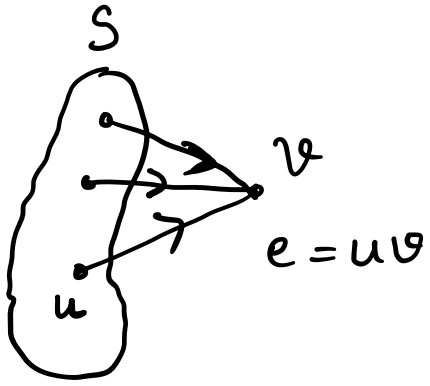
Updating the keys:



Consider an iteration in which node u is added to S, and let $v \notin S$ be a node that remains in the Priority queue.

We don't have to update $d'(v)$ if uv is not an edge, as the set of edges considered in the minimum $\min_{\substack{e=uv \\ u \in S}} d(u) + w_e$ is exactly the same before and after adding u to S.

If $uv \in E(G)$ then the new value for the key is $\min(d'(v), d(u) + w_e)$



If $d'(v) > d(u) + w_e$ then we need to use the CHANGE KEY operation to decrease the key of node v accordingly.

CHANGEKEY operation can occur at most once per edge.

Running time:

$O(m)$ + n Extract min + m Change key operations

$= O(m) + n\,O(\log n) + m\,O(\log n)$

$= O((n+m)\log n)$

$= O(m \log n)$.

($m \geq n$ as all vertices are reachable from s)

Dijkstra Using Min-Heap

```
Dijkstra(G, w, s)

    For each vertex v in V(G)
        d[v] = ∞
        v.π = NIL

    d[s] = 0

    Create an empty Min-Priority queue Q

    Q.insert(s, d[s])

    While Q ≠ ϕ
        u = Q.extract-min

        For each neighbor v of u

            if d[v] > d[u] + w(u, v)
                d[v] = d[u] + w(u, v)
                π.v = u
                If v ∈ Q
                    Q.decrease-key(Q, d[v])
                else
                    Q.insert(v, d[v])
```

Runtime Analysis

Dijkstra (G, w, s)

For each vertex v in $V(G)$ — $O(n)$

$d[v] = \infty$

$v.\pi = NIL$

$d[s] = 0$

Create an empty Min-Priority queue Q

Q. insert $(s, d[s])$ → $O(\log n)$

While $Q \neq \phi$

$u =$ Q. extract-min → $O(\log n)$

For each neighbor v of u

if $d[v] > d[u] + w(u, v)$

$d[v] = d[u] + w(u, v)$

$\pi.v = u$

If $v \in Q$ → $O(\log n)$

Q. decrease-key $(Q, d[v])$

else

Q. insert $(v, d[v])$ → $O(\log n)$

n Insertions $\longrightarrow O(n \log n)$

Extract-min $\longrightarrow$ n exactracts $= O(n \log n)$

Decrease key $\longrightarrow$ m times $= O(m \log n)$

Total running time $= O((n+m) \log n)$

$= O(m \log n)$

Correctness of Dijkstra's Algorithm

We need to show that

$$d(u) = \delta(s,u) \quad \text{for all } u \in V(G)$$

Correctness of Dijkstra's Algorithm

ie, $d(u) :=$ true shortest path distance from s to u.

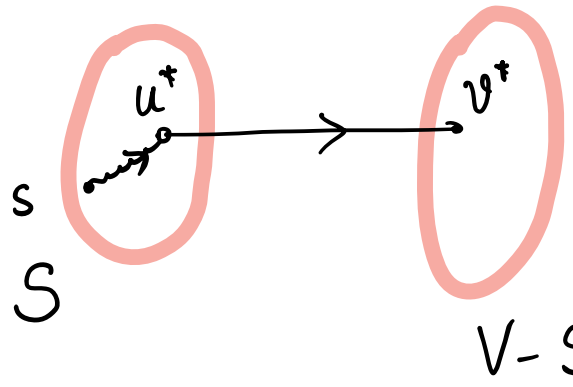
$= \delta(u)$ or $\delta(s,u)$

Proof by induction on the # of iterations (or) on size of S.

Base case: $d(s) = 0$

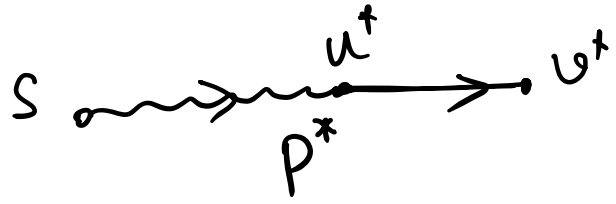
Inductive hypothesis:

For all $v \in S$, $d(v) = \delta(s,v)$



Current iteration:

We pick an edge $e^* = u^* v^*$ and add v^* to S.



By inductive hypothesis $d[u^*] = \delta(s, u^*)$

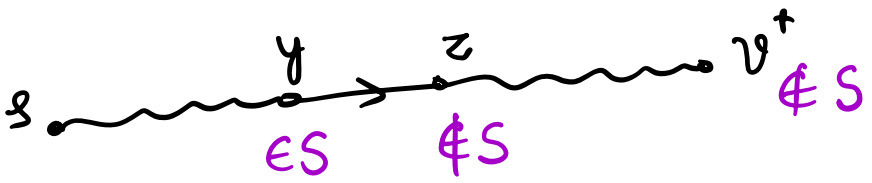
and $d[v^*] = d[u^*] + w_{e^*}$

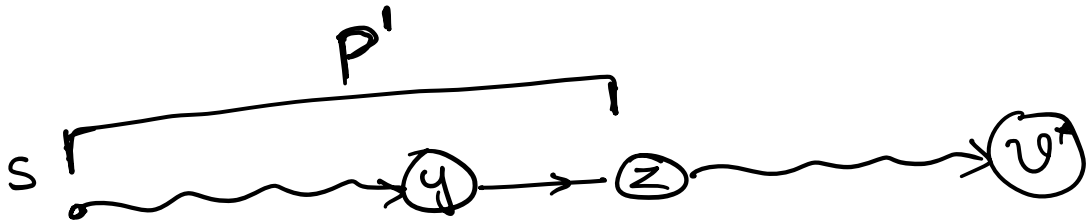
We need to Prove that

(*) length of every s-v^* Path has length $\geq d[u^*] + w_{e^*}$

Let P be any s-v^* Path.

this Path must cross S.





length of the subpath $P' \geq d[y] + w_{yz}$

By Dijkstra's greedy criterion

$$d[u^*] + w_{u^*v^*} \leq d[y] + w_{yz}$$

[the alternate s-v Path P through y and z is already too long by the time it has left the set S]

Since the edge lengths are non-negative,

the full path P is at least as long as P',

which is at least as long as P^*

$\therefore$ length of $P \geq$ length of P^*

Q: What about graphs having negative weight edges?

Graphs having negative edge weights:

(Q) Does Dijkstra's Algorithm work in this case?

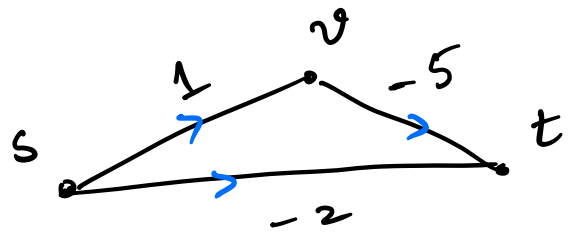
Ans See Next Page.

Idea: (to get away with negative edge weights)

Add some large Constant to each weight so that all edge weights becomes non-negative.

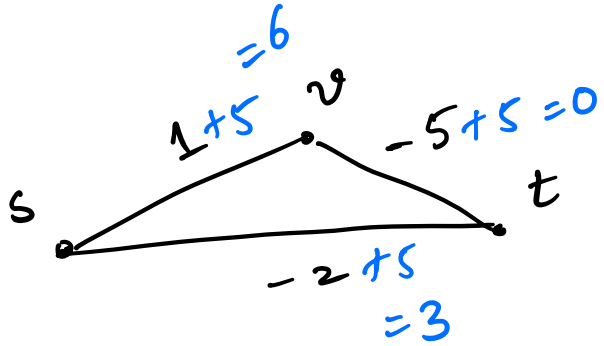
(*) But this doesn't Preserve Shortest Paths.

Example:



$d(s,t) = -4$

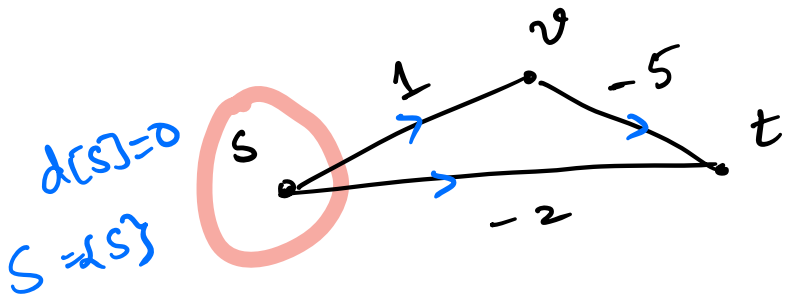
$s \rightsquigarrow v \rightsquigarrow t$



$d(s,t) = 3$

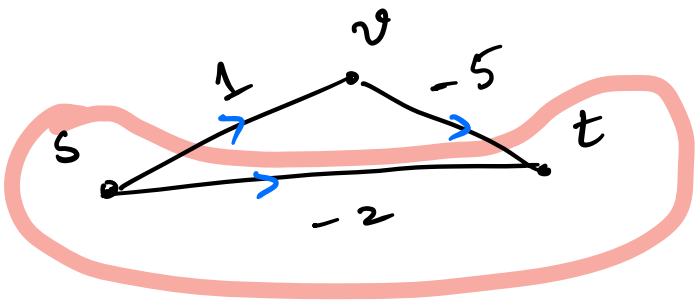
$s \rightarrow t$

Run Dijkstra's Algorithm on the same example



d[s]=0

S = {s}



d[s]=0

S = {s}

S = {s, t}

d[t] = -2

↓

This is wrong

Dijkstra fails here.